

I'm working on a small pygame "Catch the Falling Objects" game (Python 3.11, pygame 2.6). I'll give you 4 tasks one at a time. For each, change as little as possible, keep the existing structure and style, and return the full updated version of each changed file. Here are the starter files: paste collision.py, game_engine.py, basket.py, renderer.py

Yep. Send the four starter files:

- collision.py
- game_engine.py
- basket.py
- renderer.py

Then give me **Task 1**.

For each task, I'll:

- Make the **smallest possible changes**
- Preserve your existing structure, naming, and coding style
- Avoid unnecessary refactoring
- Return the **full contents of every changed file**
- Clearly indicate which files changed
- Keep it compatible with **Python 3.11 + Pygame 2.6**

the basket is supposed to catch an object only when it actually falls down to the basket, but right now an object scores the moment it spawns at the top if the basket is under its x. i think its because is_caught in collision.py only checks x and never checks y.

also the loop in update() in game_engine.py removes stuff from self.objects while looping over it, so i think it skips objects sometimes when 2 are next to each other.

can u fix both so catches need the object to be at basket height and the loop doesnt skip anything?

collision.py:

```
python
```

```

"""
collision: figures out whether a falling object
"""

def is_caught(basket_rect, obj):
    return basket_rect.left <= obj.x <= basket_r

```

game_engine.py:

```

python

"""
GameEngine: owns the basket and all falling obj
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 50
MAX_MISSES = 5

```

falling_object.py (in case u need it):

```

python

class FallingObject:
    def __init__(self, x, y, radius=14, speed=3,
                self.x = x
                self.y = y
                self.radius = radius
                self.speed = speed

```

```

        self.color = color

    def update(self):
        self.y += self.speed

    def is_past_bottom(self, height):
        return self.y - self.radius > height

```

basket.py (also in case):

python

```
import pygame
```

```

class Basket:
    def __init__(self, x, y, width=90, height=20, color=(255, 0, 0)):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.speed = speed
        self.boosted_frames = 0
        self.normal_speed = speed
        self.boost_speed = speed

```

Yep — both issues are exactly where you suspected.

- `is_caught()` now checks **both X overlap and Y position**, using the basket's vertical bounds and the object's radius so the object has to actually reach the basket.
- The catch loop now uses a **new list of remaining objects** instead of removing from `self.objects` while iterating.

Only **collision.py** and **game_engine.py** need changes. `basket.py` and `falling_object.py` stay unchanged.

collision.py

Python

```
"""
collision: figures out whether a falling object is within the basket.
"""

def is_caught(basket_rect, obj):
    x_overlap = basket_rect.left <= obj.x <= basket_rect.right
    y_overlap = (
        obj.y + obj.radius >= basket_rect.top
        and obj.y - obj.radius <= basket_rect.bottom
    )

    return x_overlap and y_overlap
```

game_engine.py

Python

```
"""
GameEngine: owns the basket and all falling objects.
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 50
MAX_MISSES = 5

class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
```

One small detail: I left the `caught` list in place to make the intent explicit, although it isn't actually needed after we build `remaining`. We can simplify that further in a later task if you want, but I haven't made unrelated changes.

the game_engine.py got cut off after `handle_input`, can u send the full file again in one code block, with all methods

(handle_keydown, update, draw too)? also send collision.py in its own code block so the formatting is right.

Yep — here are the **complete files**, separately, with nothing omitted.

collision.py

</> Python

```
"""
collision: figures out whether a falling object is within the basket.
"""

def is_caught(basket_rect, obj):
    x_overlap = basket_rect.left <= obj.x <= basket_rect.right
    y_overlap = (
        obj.y + obj.radius >= basket_rect.top
        and obj.y - obj.radius <= basket_rect.bottom
    )

    return x_overlap and y_overlap
```

game_engine.py

</> Python

```
"""
GameEngine: owns the basket and all falling objects.
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 50
MAX_MISSES = 5

class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
```

nice that works now. next thing, the basket can go half off the screen on the left and right edges. i think the boundary check in `handle_input` doesn't account for the basket's width, it just clamps `x` between 0 and `WIDTH`.

can u fix it so the basket always stays fully on screen? also i want the movement to feel smooth when i hold the arrow key down, it shouldn't jitter or get stuck at the edges. just send me the changed part of `game_engine.py` as a code block.

Yep. Since `self.basket.x` is the **center** of the basket, the limits need to account for half its width. The existing per-frame key handling already gives smooth continuous movement while the key is held, so we only need to fix the clamping.

Replace `handle_input()` with:

`</>` Python

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return

    if keys_pressed[pygame.K_LEFT]:
        self.basket.x -= self.basket.speed
    if keys_pressed[pygame.K_RIGHT]:
        self.basket.x += self.basket.speed

    half_width = self.basket.width / 2
    self.basket.x = max(
        half_width,
        min(WIDTH - half_width, self.basket.x)
    )
```

This keeps the basket's **entire width** inside the screen:

- Left edge → `x = basket.width / 2`
- Right edge → `x = WIDTH - basket.width / 2`

And because the position is clamped every frame, holding an arrow at an edge won't cause it to jitter or move partially off-screen.

cool the basket stays on screen now. next one is the spawning. right now objects spawn every 50 frames exactly and the `x` is just `random.randint`, so it feels boring and sometimes they spawn in the same place a few times in a row.

can u make it so objects spawn at different x positions and at different time gaps (not a fixed interval)? they should never spawn outside the screen, shouldnt keep spawning in the same spot, and there should be a max number of objects on screen at once so it doesnt get impossible. put the numbers in constants at the top so i can tweak them.

just send me the new constants, the new `_spawn_object` method, and the changed lines in `update()`, as separate code blocks.

Yep. We can keep the existing structure and just make spawning controlled by a few constants.

The approach:

- Randomize the **time until the next spawn** between `MIN_SPAWN_INTERVAL_FRAMES` and `MAX_SPAWN_INTERVAL_FRAMES` .
- Keep the new object's X at least `MIN_SPAWN_DISTANCE` away from the previous spawn.
- Keep the object fully inside the screen.
- Stop spawning when `MAX_OBJECTS` is already on screen.
- Use the object's radius when calculating the valid X range.

New constants

Replace the existing `SPAWN_INTERVAL_FRAMES` constant with:

Python

```
MIN_SPAWN_INTERVAL_FRAMES = 30
MAX_SPAWN_INTERVAL_FRAMES = 70

MAX_OBJECTS = 5
MIN_SPAWN_DISTANCE = 80

MAX_MISSES = 5
```

New `_spawn_object()` method

Python

```
def _spawn_object(self):
    radius = 14
    min_x = radius
```

```

max_x = WIDTH - radius

if self.objects:
    previous_x = self.objects[-1].x
    valid_positions = [
        x for x in range(min_x, max_x + 1)
        if abs(x - previous_x) >= MIN_SPAWN_DISTANCE
    ]

    if not valid_positions:
        return

    x = random.choice(valid_positions)
else:
    x = random.randint(min_x, max_x)

```

Changed lines in `update()`

Replace the current spawning section:

`</>` Python

```

self.frames_until_spawn -= 1
if self.frames_until_spawn <= 0:
    self._spawn_object()
    self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

```

with:

`</>` Python

```

self.frames_until_spawn -= 1
if self.frames_until_spawn <= 0:
    if len(self.objects) < MAX_OBJECTS:
        self._spawn_object()

    self.frames_until_spawn = random.randint(
        MIN_SPAWN_INTERVAL_FRAMES,
        MAX_SPAWN_INTERVAL_FRAMES,
    )

```

This means the game will no longer have a predictable **50-frame** → **spawn** → **50-frame** → **spawn** rhythm, while still preventing the screen from getting overloaded.

ok spawning is good now. last one, i want a speed boost for the basket. when i press space the basket should move faster for a few seconds, then go back to normal speed

automatically. there should be a clear thing on screen while the boost is on (like a timer text and maybe the basket changes color).

the Basket class already has `boosted_frames`, `normal_speed` and `boost_speed` that aren't used for anything, so you can use those. also it shouldn't be possible to spam space while it's already active.

just send me the changes as separate code blocks per file (`basket.py`, `game_engine.py`, `renderer.py`), only the new or changed methods and lines, and tell me where each one goes.

Yep. Since `Basket` already has the boost-related fields, we can use those without changing the overall structure.

I'm assuming **60 FPS**, so I'll make the boost last about **3 seconds = 180 frames**. You can tweak the constants easily.

`basket.py`

1. Add these constants at the top of the file, after `import pygame` :

Python

```
BOOST_DURATION_FRAMES = 180
BOOST_SPEED_MULTIPLIER = 2
```

2. Add this method inside the `Basket` class, after `__init__()` :

Python

```
def activate_boost(self):
    if self.boosted_frames > 0:
        return

    self.boosted_frames = BOOST_DURATION_FRAMES
    self.boost_speed = self.normal_speed * BOOST_SPEED_MULTIPLIER
    self.speed = self.boost_speed

def update(self):
    if self.boosted_frames > 0:
        self.boosted_frames -= 1
```

```
if self.boosted_frames == 0:  
    self.speed = self.normal_speed
```

game_engine.py

1. Add this to `handle_keydown()` so Space activates the boost.

Replace your current `handle_keydown()` with:

 Python

```
def handle_keydown(self, key):  
    if self.game_over and key == pygame.K_r:  
        self.__init__()  
        return  
  
    if key == pygame.K_SPACE:  
        self.basket.activate_boost()
```

2. At the beginning of `update()`, after the `game_over` check, add:

 Python

```
self.basket.update()
```

So the beginning of `update()` becomes:

 Python

```
def update(self):  
    if self.game_over:  
        return  
  
    self.basket.update()  
  
    self.frames_until_spawn -= 1
```

3. In `draw()`, change the existing score/misses section to also show the boost timer:

 Python

```
renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))  
renderer.draw_text(surface, font, f"Misses: {self.misses}/{MAX_MISSES}", (10, 36))
```

```
if self.basket.boosted_frames > 0:
    boost_seconds = (self.basket.boosted_frames + 59) // 60
    renderer.draw_text(
        surface,
        font,
        f"BOOST: {boost_seconds}s",
        (10, 62),
    )
```

renderer.py

In your `draw_scene()` method, find wherever the basket is drawn and change the basket color based on whether the boost is active.

If your basket is currently drawn with something like:

 Python

```
pygame.draw.rect(surface, (100, 180, 255), basket.get_rect())
```

change it to:

 Python

```
basket_color = (255, 215, 60) if basket.boosted_frames > 0 else (100, 180, 255)
pygame.draw.rect(surface, basket_color, basket.get_rect())
```

So the basket will **change color while boosted**, and the `BOOST: 3s → 2s → 1s` text will appear on screen.

The `activate_boost()` check prevents Space from restarting/extending the boost while it's already active.